

Weather Predictor — Tutorial de progreso

Autor: José Rubio · **Fecha:** 2026-08-16 · **Versión:** rev 6 (roster 20 · corrector de nivel · lo que se midió y se cayó)

1. Resumen ejecutivo

Weather Predictor es una aplicación educativa que predice la temperatura máxima diaria y la compara contra los precios del mercado de contratos de eventos de **Kalshi** (KXHIGHT* / KXHIGHT*) para esa misma temperatura. El scope curado son **20 estaciones**, todas las que Kalshi lista con liquidez decente: KPHX, KLAX, KLAS, KNYC, KBOS, KMIA, KMDW, KHOU, KSFO, KAUS, KDEN, KSAT, KDCA, KDFW, KPHL, KSEA, KATL, KMSY, KOKC, KMSP.

`stations.py` es la fuente única del roster: ids, serie de Kalshi, código del CLI, ventana de pico, huso y longitud. Añadir una estación es una línea ahí, y los cinco consumidores la heredan por import.

Dos ids que costaron un bug cada uno y conviene no olvidar: la estación de **Nueva York es KNYC (Central Park)**, que es donde liquida KXHIGHTNY — el viejo id "KLGA" se eliminó en el rename del 2026-07-22 y no debe reintroducirse. Y **Houston liquida con Hobby (KHOU), no con Bush (KIAH)**: Hobby corre 1-3°F más fresco en verano por la brisa de la bahía, así que no era un offset corregible sino la estación equivocada.

El objetivo no es apostar con dinero real, sino medir cuánto "edge" tiene un modelo ensemble propio frente al mercado y cómo mejora esa precisión con el paso de las horas del día.

El sistema corre 24/7 en una **Raspberry Pi 4B (8GB)** accesible vía Tailscale en `100.83.162.24`. Tres servicios en puertos separados:

- `:8000` — Weather predictor (este proyecto, las 20 estaciones)
- `:8001` — Crypto predictor (proyecto paralelo BTC quarterly, mismo enfoque)
- `:8080` — Dashboard agregado (monitor BTC, tab AI, tab de análisis)

Toda la información se persiste en SQLite para análisis retrospectivo. Polling adaptativo cada 3 min en pico, 10 min fuera.

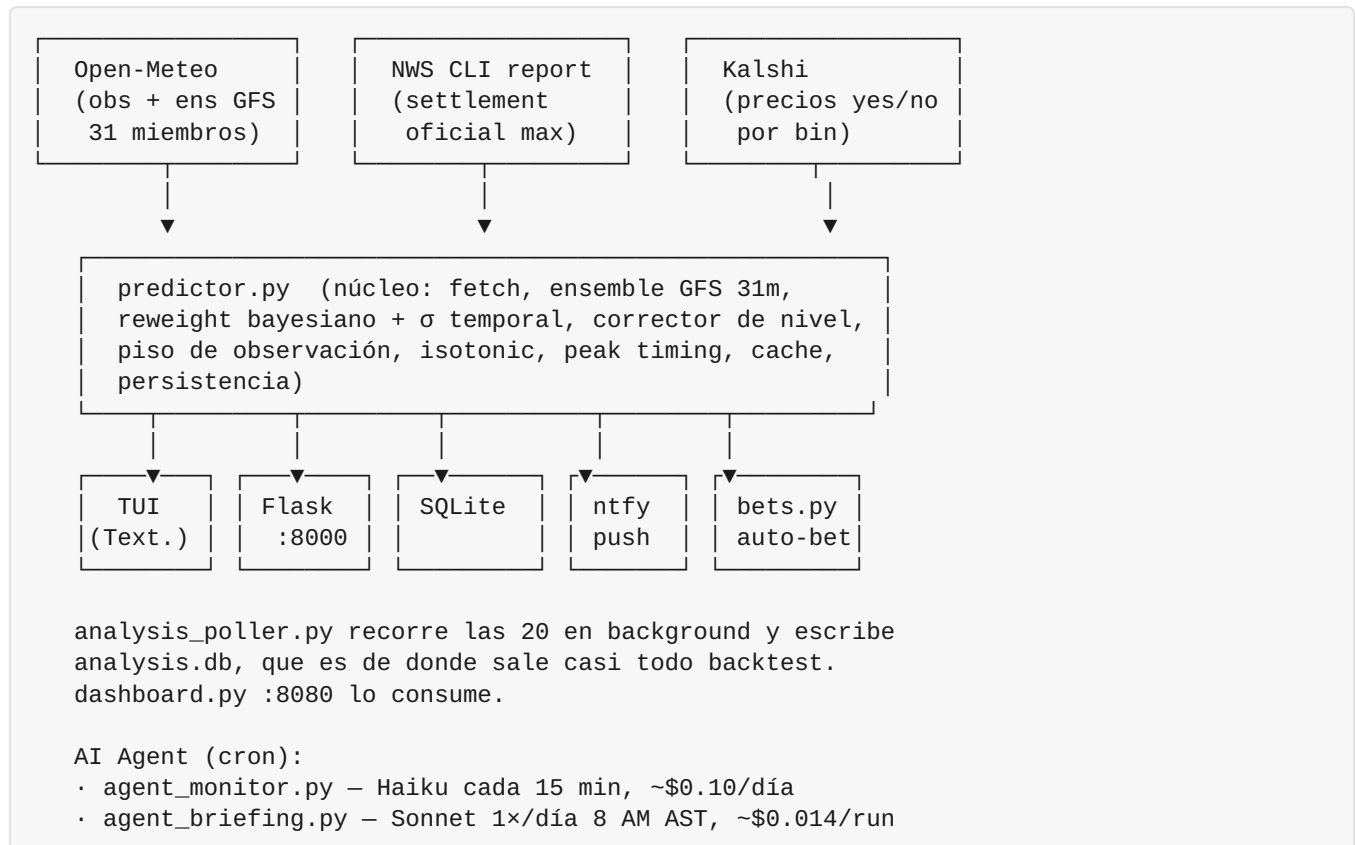
⚠ **El Pi es la fuente canónica.** Los reportes y las decisiones salen de ahí, no de la copia del repo en la laptop, que no tiene ni venv ni las bases de datos vivas.

2. Objetivo del proyecto

- **Predecir** el máximo diario en °F con una distribución de probabilidad, no un solo número.
- **Comparar** esa distribución contra los precios de Kalshi para las 20 estaciones curadas.
- **Medir** si el modelo es consistente, calibrado, y si ofrece ventaja sistemática frente al mercado.
- **Identificar** días "difíciles" (fronts, anomalías, ruptura de régimen) donde lo seguro es saltar la apuesta.
- **Aprender** sobre ensemble forecasting, calibración probabilística y prediction markets sin usar dinero real.

El diseño apunta a uso diario: el iPad muestra el dashboard :8080 vía Tailscale por la mañana, recibo el **briefing matutino** automático del agente AI a las 8:00 AM AST y decido qué estación tiene la mejor combinación de edge + dificultad baja.

3. Arquitectura



- **Polling adaptativo:** un thread de background refresca obs + ensemble cada **3 min en ventana de pico** (12–17h local según estación) y **10 min fuera**. Durante pico invalida cache de obs para leer METAR fresco.

- **Persistencia:**

- analysis.db — **la grande (~770MB):** station_snapshots (la serie que alimenta casi todo backtest), kalshi_snapshots, radar_snapshots
- calibration.db — day_outcomes (los settles), prediction_snapshots, simulated_bets, day_summary, brier_weekly, daily_skip_flags
- market_cache.db — precios mercado + nuestro p por bin
- climate_cache.db — 30 años de max diarios (Open-Meteo archive cacheado)
- agent.db — decisiones AI, budget, briefings

⚠ Asimetría que ya costó un diagnóstico entero: prediction_snapshots la llena predictor_web, que tiene **una sola estación activa a la vez**. analysis_poller escribe a analysis.db y sí cubre las 20. Cualquier feature que lea de la primera funciona sólo en la estación que el web tenga abierta — así fue como el bias EWMA estuvo dos meses aplicándose en una sola estación sin que se notara.

- **Acceso remoto:** los tres servicios visibles desde iPad/laptop vía Tailscale (`100.83.162.24`) o LAN local.
- **Auto-start:** `~/start_all.sh` arranca los 3 servicios al reinicio (cron `@reboot`).

4. Fuentes de datos

Fuente	Uso	Formato
Open-Meteo Current	Temperatura actual, dewpoint, humedad, viento, presión	API JSON, llamada cada 3/10 min
Open-Meteo Ensemble GFS	31 miembros del GFS, hora a hoy/mañana/pasado	API JSON: temperatura + precipitación + nieve
Open-Meteo Archive	30 años de max diarios por estación	API JSON, cache SQLite local
NWS Climate Report (CLI)	Max oficial del día (settle source)	Scraping texto NWS
Kalshi REST API	Precios yes/no por bin de temperatura	JSON oficial (sin auth, endpoints públicos)
Modelos externos	ECMWF/GFS/NAM/HRRR/Wunderground como sanity check	Open-Meteo seasonal API

Settle: desde el swap a Kalshi, **NWS CLI es la única fuente de settlement**. El fallback a Open-Meteo archive existe en el código pero es opt-in y está reservado (`allow_archive_fallback=False` por defecto): su max no coincide con el del CLI, y meterlo en `day_outcomes` contamina la isotónica. Un día sin CLI se queda pendiente, no se rellena.

Tres cosas aprendidas sobre el CLI, cada una a base de un settle mal puesto:

- **Vive 2-3 días** en la API. Para días más viejos se parsea el **CF6 mensual** (`fetch_max_min_cf6`), que fue como se recuperaron 300 días-estación de julio.
- El sufijo de récord (`101R`) no parseaba y hacía caer al CLI matinal: KDEN quedó settleado a 77 cuando el real era 101.
- **Nunca backtestear con `MAX(today_max_obs)` como proxy del settle**. Difiere del CLI en el 70% de los días, hasta -11°F .

El **parcial** del día en curso (`fetch_intraday_max`) sí se usa, pero sólo como piso: se consume con `max(...)` porque un parcial únicamente puede subir. Medido sobre 486 días-estación, el CLI de la tarde le gana a nuestro `today_max_obs` una mediana de $+1.0^{\circ}\text{F}$, porque el CLI mide con ASOS de 1 minuto y nuestro feed es de 5.

5. Modelo predictivo

La idea central: en vez de predecir un solo número, generar **una distribución de probabilidad** sobre cuál va a ser el máximo del día.

5.1 Ensemble GFS (baseline)

Open-Meteo ofrece los 31 miembros del ensemble GFS por separado. Cada miembro es una simulación con pequeñas perturbaciones en condiciones iniciales. Para cada miembro extraemos el max del día, dando 31 muestras de la distribución predictiva. Calculamos p10, mediana, p90 y modal directamente.

5.2 Reweight bayesiano + σ temporal

A medida que avanza el día, ya conocemos la temperatura real las primeras horas. Esa observación reweightea los miembros del ensemble: los miembros que "se parecen" a lo observado reciben más peso; los que divergieron mucho, menos.

Implementación: softmax sobre la suma de errores cuadrados (SSE) estandarizada por un σ **dependiente de la hora**:

- Pico (12-17h): $\sigma=1.5^{\circ}\text{F}$ (más exigente, más discriminación)
- Adyacentes: $\sigma=2.0^{\circ}\text{F}$
- Medias: $\sigma=2.5^{\circ}\text{F}$
- Lejanas: $\sigma=3.5^{\circ}\text{F}$

Re-muestreamos a $N=500$. El reporte muestra "eff N" (miembros efectivos tras reweight). Si $\text{eff_N} < 20 \rightarrow$ "reweight colapsado", la predicción es inestable.

5.2.1 Detector de ruptura de régimen

Si la observación de una hora cae fuera de la envolvente **p1–p99** del ensemble (con margen 4°F para absorber sesgos de estaciones urbanas), esa hora se marca como **ruptura**. Con ≥ 2 horas rotas:

- Se fuerza la dificultad del día a 100 ("ruptura de régimen")
- Push ntty al iPad
- /cross recomienda saltar el día
- El agente AI (monitor + briefing) filtra todas las opps de esa estación

Lección importante (2026-06-21): la "triple convergencia" (nuestro modelo + mediana de externos + mispricing Kalshi) **NO funciona** con régimen roto. Los modelos externos también pierden calibración en regímenes extremos — no son independientes del nuestro.

□ **Corrección de rev 6 (2026-07-27):** **difficulty** quedó revocado como gate. Sobre $N=505$ días-estación, **ninguna de sus cinco componentes predice el error** — y la de ruptura de régimen sale con el signo contrario al esperado. La regla dura de "nunca entrar si $\text{difficulty} > 70$ " ya no se aplica; el score sigue calculándose y mostrándose como color, pero no bloquea nada.

Dos cosas se descubrieron al desmontarlo: **difficulty** es un `max()`, así que la componente de anomalía lo satura ella sola (el percentil 86 ya cruza 70), y el 31% de los bloqueos de julio fueron sólo-por-anomalía. Lo que sí sobrevive del párrafo original es el detector de ruptura de régimen como **aviso**, no como veto.

5.3 Peak timing

Además del valor del máximo, predecimos **cuándo**. Para cada miembro extraemos la hora del max y reportamos modal, p10/p50/p90, P(ya ocurrió), P(ocurrirá en próximas 1/2/3/6 horas).

5.4 Corrector de nivel (sustituye al bias tracker EWMA)

La idea original era un EWMA del bias por estación: si sobrepronostica +2°F de forma persistente, bajar la mediana. **El EWMA quedó jubilado el 2026-08-14** por dos razones independientes:

1. Acertaba la dirección el **49.7%** de los días — azar puro. Al ponderar lo reciente sobre 4-5 muestras, un solo día de ruptura lo volcaba: en KPHX, cuatro muestras negativas y un +8.91 dejaron el bias en +3.29, restando cuando había que sumar.
2. Su fuente (`prediction_snapshots`) sólo se llenaba para la estación activa del web, así que **19 de 20 estaciones tenían cero filas** en su ventana matinal. Nunca fue una corrección del roster.

Lo sustituye el **corrector de nivel** (`level_corrector.py`): resta la **mediana causal** de los sesgos de días anteriores de esa estación. La mediana es robusta al día de ruptura por construcción, y el sesgo por estación resulta ser estable (Spearman entre primera y segunda mitad = +0.70). Medido sobre 460 días-estación: sin corregir 2.00°F, EWMA 1.94°F, **mediana causal 1.31°F**.

Dos detalles que importan:

- Es **por hora local**, porque el sesgo decae según avanza el día y el ensemble incorpora observaciones. En KNYC va de +4.00°F a las 11h a +0.07°F a las 22h. Aplicar el sesgo matinal por la tarde sobre-corregía más de 2°F.
- **Cada estación entra con su propio backtest pre-registrado**, nunca por extrapolación del pool. Hoy: KLAX y KSFO (2026-08-05), KNYC (2026-08-15). KDCA es el contraejemplo que se vigila: allí el corrector empeora.

5.4.1 Piso de observación

Invariante que parece obvio y que el código rompía: **la predicción no puede estar por debajo de lo que el termómetro ya marcó**. Se encontró en 174 de 176 casos que era error garantizado — el invariante existía, pero los post-ajustes (bias, isotónica) lo pisaban después. `apply_obs_floor` lo reimpone al final, y el corrector tiene su propia guarda (`cap_by_floor`) para no hundir la mediana bajo el piso dentro de la ventana de pico.

5.5 Climatology anchor

En heatwave (`ext_diff` $\geq +1.5^\circ\text{F}$ en oeste/sur o $p \geq 85\%$ de cola alta), anclamos a la mediana de modelos externos en vez de NO-sellar bins altos. Memoria de fallos previos confirmó que en heatwave los externos ganan al modelo crudo.

5.6 Calibración isotónica

PAV (Pool-Adjacent-Violators) sobre snapshots históricos (`predicted_p`, `outcome`). Gate actual: ≥ 20 pares y ≥ 7 días distintos. Cuando pasa el gate, se aplica en vivo a ladder/comparison. Estado actual: variable según estación.

6. Integración con el mercado (Kalshi)

Desde el swap de 2026-05-08 usamos Kalshi como mercado en lugar de Robinhood. Kalshi tiene API REST oficial (sin auth para market data), bins definidos por rango de temperatura (`[lo, hi]` , con tails `$\leq X$` y `$\geq Y$`),

y settlement alineado con NWS CLI — exactamente lo que necesitamos para no tener divergencia de fuentes.

Series tracker: una por estación, todas en `stations.py` (`KXHIGHTPHX` , `KXHIGHTLAX` , `KXHIGHTLV` , `KXHIGHTNY` , `KXHIGHTBOS` , `KXHIGHTMIA` , `KXHIGHTCHI` , `KXHIGHTHOU` , `KXHIGHTSFO` , `KXHIGHTHAUS` , `KXHIGHTDEN` , `KXHIGHTSATX` , `KXHIGHTDC` , `KXHIGHTDAL` , `KXHIGHTPHIL` , `KXHIGHTSEA` , `KXHIGHTATL` , `KXHIGHTNOLA` , `KXHIGHTOKC` , `KXHIGHTMIN`).

6.1 `/comparison` — Kalshi vs modelo

Tabla bin por bin con `yes_mid` de Kalshi y nuestra probabilidad post-calibración (bayes + bias + isotonic). Barra visual y diff en pp. Esta es la **predicción final** del modelo, la que usa el bot para auto-bets — no confundir con el `/analysis` del dashboard `:8080` que muestra el ensemble crudo.

6.2 `/ladder` — Decision ladder

Para cada threshold ($\pm N$ alrededor de mediana):

- `yes/no` nuestro vs `yes/no` Kalshi
- `edge` en puntos porcentuales
- EV al apostar \$1: $EV_{yes} = (p-k)/k$, $EV_{no} = (k-p)/(1-k)$
- Kelly fraction: $f_{yes} = (p-k)/(1-k)$, $f_{no} = (k-p)/k$
- Pill **YES/NO** señalando lado recomendado

6.3 Edge y movement (rutas absorbidas)

- **Edge tracking** — ahora en `/comparison?sort=edge` : contratos con mayor edge actual + análisis histórico (edges por bucket, ROI hipotético).
- **Movement** — ahora en `/intraday` : evolución temporal de `yes_mid` Kalshi vs `our_p` para un bin dado, junto al peak timing.

6.4 `/bets` — Simulador P&L

Cuando `|edge| ≥ 5pp` , el sistema "apuesta" \$10 hipotéticos en el lado correcto (guard contra precios ≤ 0.01 o ≥ 0.99). Al settlearse el día, calcula payoff real. Filtros anti-pérdidas vigentes (desde 2026-05-26):

- Bloqueo por bias o losing streak
- Mid-bin via `our_pred` (no apostar bins muy estrechos)
- Divergence overnight con externos
- Auto-cleanup de bets erróneas

Dedupe por `(station, date, ticker)` .

6.5 Tail-negation — ⚠ RETIRADA

La regla decía: con modelo sesgado, preferir NO $[\leq X]$ de cola con `edge ≥ 40pp` sobre point-bets centrales, validada sobre un caso de mayo que ganó +\$7.50.

Se invalidó el 2026-07-07. La auditoría del ledger mostró que su validación vivía sobre datos rotos — el mismo defecto que inflaba el ROI global a +53% ficticio. La UI se retiró. No readmitirla sin $N \geq 100$ posterior al fix.

6.6 Guarda de techo físico

Añadida el 2026-08-14 (`physical_gate.py`) tras varios días en que el gate marcaba ACTIONABLE contra lo que el termómetro ya decía. Acota el máximo plausible del día por tres vías, de dura a blanda: pico confirmado o ventana cerrada → `max_obs + 1.0` ; meseta larga con poca ventana → `max_obs + 2.0` ; resto → `max_obs` + el p90 histórico de subida restante a esa hora.

Con eso veta dos cosas: **YES** sobre un bin cuyo suelo supera el techo (no puede ganar), y **NO** sobre el bin donde cae el techo (es el desenlace más probable). La tercera vía usa el p90 a propósito, así que es generosa y rara vez vetará: la guarda es fuerte cuando hay observación dura y casi inerte el resto del tiempo.

6.7 `/calibration` + `/bets?view=history`

- **Calibration**: reliability diagram dual (nuestro vs Kalshi) + Brier per-bin.
- **History** (tab dentro de `/bets`): tabla diaria con max observado, Brier nuestro, Brier Kalshi, mejor edge, si fue correcto. Total días ganando al mercado.

6.8 Push notifications (ntfy)

Opt-in vía env `NTFY_TOPIC`. Edge ≥ 10 pp → push al iPad. Settle → push con Briers comparados. El briefing matutino del AI agent también puede empujar a ntfy si está configurado.

7. Guía por página

La web `predictor.py` corre en `http://100.83.162.24:8000`. El dashboard agregado en `:8080`. Crypto en `:8001`.

Para decidir si apostar hoy

- `/` (**dashboard principal :8000**) — Todo del día en una pantalla: temp actual, pronóstico, distribución del ensemble, badge de dificultad (informativo, ya no bloquea), **dropdown con las 20 estaciones** sin reiniciar, cards de clima/viento/presión, peak timing, precipitación, aserciones personalizadas. **Línea "edge máx" arriba indica la decisión final del bot**. Avisos que pueden salir aquí: `MAX OBS CONGELADO` (el METAR horario falta pero el feed sigue) y `OBSERVACIÓN VIEJA` (la estación calló del todo — son modos de fallo distintos y hacen falta los dos).
- `/cross` — Las estaciones lado a lado con **ranking** y **pill de recomendación**: "apuesta a STATION SIDE +Xpp" (#1 con edge ≥ 5 pp y dificultad < 30) o "⚠ mejor saltar hoy". Selector hoy/mañana/pasado.
- `/comparison` — Barra visual bin-por-bin: yes_mid Kalshi vs nuestra prob **post-calibración** + diff en pp.
- `/ladder` — Por temperatura umbral: nuestro yes/no vs Kalshi, edge en pp, EV%, Kelly, pill YES/NO. Selector ventana $\pm 2/\pm 3/\pm 4/\pm 6/\pm 10$ /todo.
- `/comparison?sort=edge` — Toggle "orden: edge" en `/comparison`: edges actuales + performance histórica por bucket (colapsable).

Dashboard agregado :8080

- `/analysis` — las 20 estaciones, bins Kalshi vs `our_p` raw, tabla de aseveraciones del user con probabilidad declarada. `our_p` aquí es **ensemble crudo (count_in_bin/N)** — no aplica bayes/corrector/isotónica. Para predicción final ir a `:8000/comparison`.
- `/btc-quarter` — Monitor BTC quarterly (proyecto paralelo, ver crypto predictor).
- `/ai` — Tab del AI agent: budget, gasto del día, briefing matutino destacado arriba, tabla de decisiones del monitor.

Para entender el clima

- `/intraday` — Hora del pico (modal, rango p10-p90, P(ya ocurrió), P(en próximas 1/2/3/6h)) + evolución temporal Kalshi vs nuestro pronóstico.
- `/precip` — Probabilidad lluvia/nieve hoy/mañana/pasado.

Para ver si el sistema funciona

- `/reweight` — Diagnóstico hora por hora del reweight: obs vs ensemble p10/p50/p90, σ aplicado (verde=pico), n miembros matched, flag dentro/fuera p1–p99. Rupturas en rojo.
- `/calibration` — Reliability dual + Brier crudo vs calibrado.
- `/bets` — Simulador P&L: n bets, win rate, stake, payoff, ROI. Tab "history" da tabla diaria + Brier nuestro vs Kalshi.

Utilidades

- `/status` (salud), `/about` (este tutorial), `/export` (CSV de 5 tablas), `/notify`. Bookmarks viejos (`/edge`, `/timing`, `/movement`, `/history`) redirigen 301 preservando query string.

8. AI Agent

Componente añadido en 2026-06-21. Dos procesos cron-driven que usan la API de Anthropic Claude:

8.1 `agent_monitor.py` (cada 15 min, Haiku 4.5)

Lee `analysis.db` (snapshots + bins de las 20 estaciones) + aseveraciones del usuario, manda contexto estructurado a Claude Haiku con un system prompt que codifica:

- Reglas duras: spread $\leq 5^\circ\text{F}$, conviction tiers, lógica de side correcta
- Detección de mercado settled (ens_spread=0 + obs $\approx$ ens_med)
- Triple convergencia (modelo + ext_med + mispricing) $\triangle$ con el gate de `difficulty` revocado desde 07-27, ver §5.2.1

El cron dispara cada minuto pero el script decide si corre según `interval_min` en la DB (15 por defecto, 1 en modo burst); así se cambia la cadencia sin tocar el crontab.

Output JSON con oportunidades (conviction high/med). Guarda en `agent.db`. Costo medido: ~\$0.001/call → ~\$0.10/día. Budget cap soft \$14.50; auto-pausa al alcanzarlo.

8.2 `agent_briefing.py` (8:00 AM AST, Sonnet 4.6)

Lee snapshots actuales + outcomes de ayer + última decisión del monitor. Genera briefing narrativo de 6-8 líneas con setups del día, estaciones a evitar, lección de ayer. Se muestra al tope de `/ai`. Push ntfy opcional.

Costo medido: ~\$0.014/call → ~\$0.42/mes.

8.3 Control y visibilidad

- Tab `/ai` en dashboard: budget cap, gasto total, gasto hoy, proyección mensual, estado (activo/ pausado con botón toggle), briefing destacado, tabla de últimos 20 ciclos con opps detectadas y razonamiento.
- Hard cap definible en Anthropic Console por si el soft cap falla.
- Lecciones aprendidas se guardan como memorias del usuario (`feedback_*.md`) y se inyectan en el system prompt de futuras corridas.

9. Decisiones abiertas

1. **Extender el corrector de nivel.** Hoy KLAX, KSFO y KNYC. El exploratorio apunta a KMIA, KMSP, KMSY y KSEA como siguientes; KOKC, KLAS y KBOS empeoraban. Cada una pide su backtest pre-registrado.
2. **Cross-station Bayesian reweight.** Usar obs matutinas de KEWR/KJFK para reweightear KNYC. No implementado.
3. **Al agent fase 2 — chat endpoint `/ask`.** Conversación con el modelo sobre estado actual (usaría Sonnet, ~\$0.03/pregunta).
4. **Al agent fase 3 — post-mortem automático.** Al settlearse el día, agente compara predicción vs outcome y guarda lecciones en `agent_lessons.db` que se inyectan al prompt del día siguiente.
5. **Más features ensemble.** Ya: lluvia/nieve. Falta: viento max, nubosidad, ráfagas.
6. **Gate isotónica.** Hoy ≥ 20 pares, ≥ 7 días. ¿Bajar a 5 días para activar antes?

10. Próximos pasos por ROI

Completados desde rev 3 (abril → junio 2026):

- ✓ Swap Robinhood → Kalshi (2026-05-08), settle a Central Park
- ✓ Bias tracker EWMA por estación
- ✓ Climatology anchor en heatwave
- ✓ Detector de divergencia con modelos externos + card en dashboard
- ✓ KMIA + KMDW añadidos al scope curado
- ✓ Filtros anti-pérdidas (bias/streak block, mid-bin guard, divergence overnight, auto-cleanup)
- ⚠ Tail-negation rule — documentada en rev 5, **retirada el 07-07** (ver §6.5)
- ✓ Settle exclusivo NWS CLI (sin fallback Open-Meteo)
- ✓ Codex review 2026-06-18 — P0+P2 aplicados, P1/P3 pendientes
- ✓ Crypto predictor paralelo en `:8001` (BTC quarterly)

- ✓ Migración a Raspberry Pi 4B (2026-06-19) — 100.83.162.24, auto-start, 24/7
- ✓ Dashboard agregado :8080 con /analysis , /btc-quarter , /ai
- ✓ AI agent (monitor 15min + briefing matutino) deployado 2026-06-21

Añadido en rev 6 (julio → agosto 2026):

- ✓ Roster ampliado de 7 a **20 estaciones**, con `stations.py` como fuente única
- ✓ Rename KLGA → KNYC y fix de Houston (KIAH → KHOU), ambos verificados contra el settle real de Kalshi
- ✓ Auditoría del ledger (07-06/07): el ROI de +53% era artefacto; 5 fixes y modo seguro
- ✓ Parser CF6 + backfill de 300 días-estación de settles perdidos
- ✓ Piso de observación y CLI-first intradía
- ✓ Corrector de nivel en KLAX, KSFO (08-05) y KNYC (08-15)
- ✓ Guarda de techo físico y aviso de `OBSERVACIÓN VIEJA`
- ✓ EWMA del bias jubilado

Pendientes:

1. **Cross-station reweight** (obs matutinas de estaciones cercanas).
2. **mid-YES con `ext_diff` alto** — pide $N \geq 100$ y un control nuevo: el que se usó era el complemento aritmético de la misma apuesta y no falsaba nada.
3. **Medir el efecto del clamp del piso**, vigilando que el colapso de la distribución no dispare bets.
4. **HRRR/AIFS** como modelos adicionales (NBM ya aplicado).
5. **Radar dBZ** vía Iowa Mesonet NEXRAD → gate anti-convección.
6. **AI agent /ask chat** — endpoint de conversación con Sonnet.
7. **Post-mortem agent** — lecciones automáticas inyectadas al prompt.

11. Lo que se midió y se cayó

La sección más útil del documento, porque cada línea costó un backtest. Todo esto **sonaba bien y no funciona**:

Hipótesis	N	Resultado
<code>difficulty</code> como gate de entrada	505	Ninguna componente predice el error; la de régimen sale al revés
Nubosidad / radiación solar	584	$\rho = +0.015$. El GFS ya las incorpora
Advección (viento de mar)	540	No explica qué día fallamos; es offset por estación
Humedad del suelo	428	El signo va contra Bowen. El GFS ya la incorpora
"Los edges vienen de distribución difusa"	19/20	Falso; el Brier de Kalshi nos gana 7 de 9
Ventaja de 11 min sobre el precio de Kalshi	—	Era un tick de muestreo. El mercado lidera 2:1, $p \approx 0.004$
Tail-negation con edge $\geq 40pp$	—	La validación vivía sobre el ledger roto
Fallback al feed de 5 min	—	Cierra 94% del gap pero rompe el piso el 29% de los días

Y las reglas de método que salieron de ahí, que valen más que cualquiera de los resultados:

- **Pre-registrar el criterio y commitarlo antes de correr** el backtest.

- **Decidir con rho DENTRO de estación + test de signos.** El pool cruzado exagera: 2 de 3 backtests habrían dado falso positivo con él.
- **Días frescos.** La base de julio lleva doce pasadas; buscar en ella otra vez es encontrar algo por azar.
- **N<10 no cambia un threshold.**
- Lo que depende de **bets ejecutadas** (ROI real, si el edge existe) necesita **meses**, no semanas. Lo que depende de `station_snapshots` acumula rápido.

Contexto honesto para leer todo lo anterior: en los cierres diarios comparando la predicción del mediodía contra el settle, **el mercado nos gana con regularidad** (15-5, 13-7, 13-4 en tres días de agosto). El proyecto mide cuánto edge hay, y la respuesta hasta ahora es que poco.

Apéndice: métricas mencionadas

- **Brier score:** error cuadrático medio entre prob predicha y outcome (0/1). Menor = mejor.
- **Reliability diagram:** por bucket de prob predicha, qué fracción ocurrió. Si cae en diagonal, calibrado.
- **Kelly fraction:** fracción óptima del bankroll para maximizar crecimiento log esperado.
- **Edge en pp:** diferencia entre tu prob y la del mercado. Positivo = mercado subestima.
- **eff N:** miembros efectivos del ensemble tras reweight bayesiano. Bajo respecto a 31 = ensemble no capturó la realidad reciente.
- **Difficulty 0-100:** score combinado (spread + eff_n + clim + precip + regime). $\triangle$ Informativo desde 07-27, ya **no bloquea entries** — ver §5.2.1. Es un `max()`, así que la componente de anomalía lo satura sola.
- **Ruptura de régimen:** ≥ 2 horas con obs fuera de p1–p99 del ensemble. Fuerza difficulty=100, bloquea entries.
- **Conviction tier (AI agent):** high = edge ≥ 30 pp + 3 señales convergen + difficulty ≤ 70 ; med = 15-30pp; low no se reporta.